

Activation Sparsification in Transformers: Pressure, Thresholding, and Site Placement

Working draft — methods and experimental results — 8 September 2026

1 Introduction

Activation sparsity can improve language-model inference efficiency when specialized kernels skip computation and memory access associated with zero activations. Recent work has shown that sparsity can be exposed in pretrained models through post-hoc thresholding or induced during training, in both cases with measured inference speedups (Liu et al., 2024; Cetin et al., 2026). These results raise a practical design question: how should a researcher place and combine sparsification interventions to obtain more useful sparsity at an acceptable quality cost? Answering this question matters when choosing which components of a recipe to retain, where to invest additional training, and which operations justify sparse-kernel development.

Existing approaches induce activation sparsity in different ways. TEAL applies post-hoc thresholding to pretrained models, ProSparse uses L1 regularization on FFN activations during training, and Q-Sparse applies top-k sparsification to both FFN and attention activations (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). While each approach is effective in its own setting, it remains unclear how they compare under a common setup and how their effects interact when combined. An intervention that works well in isolation may add little on top of another or incur a different quality cost when applied at additional sites, making sparsity alone insufficient to identify which choices drive the improvement.

We study this problem through three separately controllable design choices: *pressure*, which encourages selected activations toward zero; *thresholding*, which sets values to exactly zero according to a gate; and *site placement*, which specifies where each acts. *The interventions are separately controllable, but their effects need not be separable.* Our central question is how their individual effects and interactions determine the quality–sparsity trade-off. In particular, does adding pressure become more or less useful as thresholding changes, and how does that response depend on where pressure and gates are applied?

Explaining that response requires connecting learned behavior to the computation it affects. A reduction in activation magnitude can leave the number of exact zeros unchanged; additional zeros can also be concentrated at sites that affect only a small share of the forward pass. Conversely, extending the gate set can expose additional operations to sparsification while changing the representation the model must learn. The empirical issue is how learning responds to these choices: when pressure produces additional zeros, how much quality that change costs, and which operations account for the result. Distinguishing these outcomes helps diagnose whether limited gains reflect the response to pressure, the computational reach of the chosen sites, or the cost of expanding the intervention.

To make that diagnosis model-wide, we pair observed zero-product counts with an architecture-dependent reach ceiling. $\mathcal{S}_{\text{model}}$ measures the fraction of counted forward matrix-multiplication products with a zero activation operand; $\mathcal{S}_{\text{model}}^{\text{max}}$ describes the fraction reachable if the selected sites were entirely zero. We refer to the former as *model-wide sparsity*, distinguishing it from a local activation zero fraction. This accounting makes the location of sparsity consequential: the same local zero rate can affect different amounts of computation as the architecture or workload changes. It helps distinguish a change in learned sparsity from a change in the work reachable through the chosen sites. Direct timing then tests whether an implementation can exploit the observed zeros on hardware.

We organize matched pretraining comparisons around a *sparsification intervention ladder* (Figure 6), which organizes gate and pressure alternatives across feed-forward (FFN) and attention sites. Pythia-family models provide a common architecture for examining these choices and their behavior across sizes (Biderman et al., 2023). We use the ladder to trace individual and joint intervention

effects from learned activations to the operations they affect, then test selected recipe relationships at larger sizes where both learned responses and computational reach can differ.

The results show that pressure’s additional value depends on the gate setting and target set. Broader interventions reshape attention as well as FFN distributions, and the high-threshold seven-site recipe improves both quality and sparsity over its four-site counterpart at all three sizes. Direct timing closes the comparison: the selected inference implementation achieves a $1.234\times$ geometric mean speedup across 30 Pythia-14M checkpoints, while ablations show that fusion and sparse-path overhead both matter. Together, these findings distinguish learning a sparse representation, reaching substantial model-wide work, and executing that work efficiently.

2 Related Work

2.1 Sparsification interventions

Post-hoc thresholding reveals sparsity that an already learned representation can tolerate. CATS thresholds small-magnitude activations within gated FFNs, with both training-free and fine-tuning settings (Lee et al., 2024). TEAL extends magnitude thresholding to attention and FFN projections and uses block-wise greedy allocation to account for their different sensitivities (Liu et al., 2024). Its placement and allocation studies show why a uniform sparsity target need not be equally appropriate throughout a model. These methods provide a reference for sparsity available without retraining, while motivating the question of how training can change that tolerance.

Training-time interventions allow the representation to adapt to sparse computation. ReLU Strikes Back studies restoring sparse activations in language models (Mirzadeh et al., 2023); ProSparse combines ReLU, progressive ℓ_1 pressure, and threshold shifting during continued training (Song et al., 2024). ProSparse’s schedule ablations show that comparable FFN sparsity can carry different quality costs. Sparsing Law examines how sparsity depends on regularization, data, and model size (Luo et al., 2025), while Cetin et al. (2026) demonstrate highly sparse FFNs induced by ℓ_1 regularization during pretraining. Together, these studies establish pressure as a way to reshape the quality–sparsity trade-off. A local FFN zero fraction alone leaves a further accounting question: how much of the complete model’s computation do those zeros affect?

Extending sparsification beyond FFNs makes the choice of sites explicit. Q-Sparse applies top- k selection to attention and FFN projection inputs, reports overall sparsity and activated-parameter budgets, and ablates both the sparsifying nonlinearity and the straight-through estimator (Wang et al., 2024). Spark Transformer combines sparse FFNs with sparse attention through token-position selection and examines the role of its low-cost predictor (You et al., 2025). Projection-input sparsity and attention-position sparsity affect different work; our intervention ladder also considers feature coordinates in the internal attention operands. Building on the existing component studies, we examine how the additional effect of activation pressure depends on thresholding and placement. Relating those effects to model-wide product counts and topology-dependent reach connects the quality cost of an intervention to the computation its zeros can affect.

2.2 Inference speedup

Converting activation sparsity into speedup requires an execution strategy suited to the workload. TEAL accelerates batch-one autoregressive decoding, where skipping weight accesses is valuable, and reports up to $1.8\times$ speedup at 50% sparsity under its projection-based accounting (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication: their TwELL format packs sparse activations as a projection produces them and supports fused FFN computation, reducing the overhead of materializing and consuming the sparse representation. They evaluate throughput for full-sequence forward passes, a different workload from token-by-token decoding. Spark likewise cou-

ples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). These results show why the location and execution of sparsity matter together. Skipped products and memory traffic must repay indexing, packing, and launch overheads, while operations that remain dense limit the gain of the complete model. A sparsity percentage alone does not specify this balance.

This dependence makes kernel adaptation part of evaluating whether a sparsification method transfers. With larger batches, different token masks can reduce the weight-loading savings available from sparsity, as Spark’s batching analysis illustrates (You et al., 2025). Block Q-Sparse introduces structure into the mask to accommodate batched execution (Wang et al., 2024); TwELL instead makes packing and fusion compatible with tiled computation (Cetin et al., 2026). Tensor shape, sparsity structure, and data layout therefore matter alongside the zero fraction. Iterative kernel generation with execution and profiling feedback offers an emerging tool for exploring such adaptations (Ouyang et al., 2025; Luo et al., 2026). Establishing a benefit still requires numerical correctness and complete-model timings that include selection and conversion costs on the target workload. These measurements test whether intervention-induced zero-product opportunities translate into reduced execution time after sparse-execution overheads are included.

3 Methodology

3.1 Sparsification interventions

A sparsification intervention specifies a gate-site set $\mathcal{G}$, gate families and thresholds at those sites, and a pressure method with target set $\mathcal{P}$. Gate and pressure sites are independently specified: the former determine where values are set to zero, and the latter where an auxiliary objective encourages small activations.

We use ReLU and two fixed-threshold gates: $G_{+,\kappa}(x) = x\mathbf{1}\{x \geq \kappa\}$ and $G_{\pm,\kappa}(x) = x\mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. One-sided thresholding removes negative values and small positive values; symmetric thresholding removes small magnitudes while preserving the signs and values of survivors. At $\kappa = 0$, their forward maps are ReLU and the identity, respectively. Appendix A specifies boundary and derivative conventions.

Pressure uses $\mathcal{L}_1$, the equally weighted mean of the mean absolute activations of each targeted site-layer tensor, taken after its gate when one is present. *Naive LI* optimizes $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$. *Orthogonal LI* (OL1) makes a task-only AdamW update, then adds a pressure correction preconditioned by the task’s second-moment estimates. When this correction opposes the adaptive task direction, OL1 projects out its conflicting component. A positive budget caps its norm relative to that task direction. This controls update geometry without guaranteeing preserved task loss; Appendix A.2 defines the procedure.

3.2 Sparsity and architectural reach

Local sparsity is the fraction of exactly zero entries at a named site. To weight zeros by the work they affect, we define *model-wide sparsity* as $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, reported as $100\mathcal{S}_{\text{model}}\%$. Here, N_0 counts scalar multiplications with at least one zero activation operand in the declared transformer-block matrix products. N_{model} counts all valid multiplications in those operations and the dense language-model head; the head receives no zero credit. Counts are pooled before division. The percentage refers to this declared multiplication workload, not all operations or elapsed time. We count only causally valid pairs in a full, uncached sequence; Appendix B gives the exact counters.

The *all-zero reach ceiling*, $\mathcal{S}_{\text{model}}^{\max} = N_{\text{reach}}(\mathcal{G})/N_{\text{model}}$, counts the fraction of that workload whose zero operand is guaranteed by setting the selected gate sites entirely to zero, crediting each affected multiplication once. It depends on the architecture, site set, and sequence length, but not pressure

or a checkpoint. The ceiling characterizes selected-site reach rather than sparsity achievable at acceptable quality: observed $\mathcal{S}_{\text{model}}$ also includes natural zeros outside that reach. Appendix B.2 defines the reach accounting.

4 Experimental study

We examine how pressure, thresholding and placement change quality and model-wide sparsity, which recipe relationships persist across model sizes, and whether specialized kernels can exploit the resulting zeros. We pretrain randomly initialized Pythia-14M, 70M, and 410M models (Biderman et al., 2023) on MiniPile (Kaddour, 2023). Their shared parallel attention and FFN architecture gives the sites a consistent interpretation across sizes. Detailed comparisons use 14M; the larger models cover selected recipes. We pair validation cross-entropy and $\mathcal{S}_{\text{model}}$ from the same checkpoint and forward configuration. Appendix C specifies the architecture and evaluation details.

The architecture map in Appendix C.1 (Figure 6) locates the normalized attention and FFN inputs (a, m), FFN hidden activation (h), concatenated attention context (z), and query, key, and value operands (q, k, v). Query and key gates follow rotary position encoding (RoPE); z precedes the attention-output projection. A0 retains the stock block. A1-H replaces GELU at h with ReLU; A4 applies one-sided thresholding at $\{a, m, h, z\}$; A7 adds symmetric gates at $\{q, k, v\}$. The L1/OL1 variants apply pressure to all gated sites in their row. These are separately pretrained conditions, not stages of a training curriculum. The ladder includes each recipe’s analytic reach, which changes with model architecture.

Within each size, matched contrasts share initialization, data order, and training budget. Adding pressure at fixed gates measures its additional quality and sparsity effects; repeating the contrast across thresholds tests how those effects depend on thresholding. Comparing A4 with A7 without pressure examines broader gate placement. A4-OL1 versus A7-OL1 changes both gate and pressure sets, so it compares complete recipes rather than isolating gate placement at a fixed pressure objective. At 70M and 410M, we evaluate A0, A1-H, A4-OL1, and A7-OL1 with the same training-token budget as at 14M. Comparisons use one initialization seed per size; thresholds are treatment levels, not statistical replicates. The cohort contains 30 conditions at 14M and 12 at each larger size. Each trains for 712 updates and 1.493 billion input tokens; all endpoints are measured at the final update. Appendix C.4 gives the realized optimizer, precision and learning-rate settings. Across sizes, we examine each operation’s zero-product fraction and its share of the model workload to interpret changes in $\mathcal{S}_{\text{model}}$ alongside $\mathcal{S}_{\text{model}}^{\text{max}}$.

For the evaluation-only reference, we apply uniform magnitude-clipping targets $p = 0, 0.1, \dots, 0.9$ at a, m, h, z without updating weights. Site-layer thresholds are calibrated separately on ten training blocks. This is a uniform TEAL-style control (Liu et al., 2024), not a reproduction of an optimized allocation across sites. The main figures show A0 clipping; Appendix D.3 includes all 540 evaluations across the 54 trained checkpoints. All training results use one initialization seed per size and complete validation, as defined in Appendix C.3.

4.1 Quality–sparsity trade-offs

The 14M results occupy distinct quality–sparsity regimes (Figure 1). Local OL1 at h can improve validation loss while introducing a modest amount of model-wide sparsity: at $\lambda = 0.5$, A1-H-OL1 reaches 3.768% sparsity at loss 5.1102, compared with A0’s loss of 5.2086. A0 post-hoc clipping reaches a similar sparsity level, 3.832% at target $p = 0.3$, but with loss 5.3619. Thus, the trained representation and evaluation-only clipping offer different quality costs even near the same aggregate sparsity.

Broader interventions reach a different regime. A7-OL1 at $\kappa = 0.5$ attains 27.483% model-wide sparsity at loss 5.8294, accepting a quality cost relative to A0. This comparison also changes compu-

Quality vs. model-wide sparsity frontier for
train-time and post-hoc interventions (Pythia-14M)

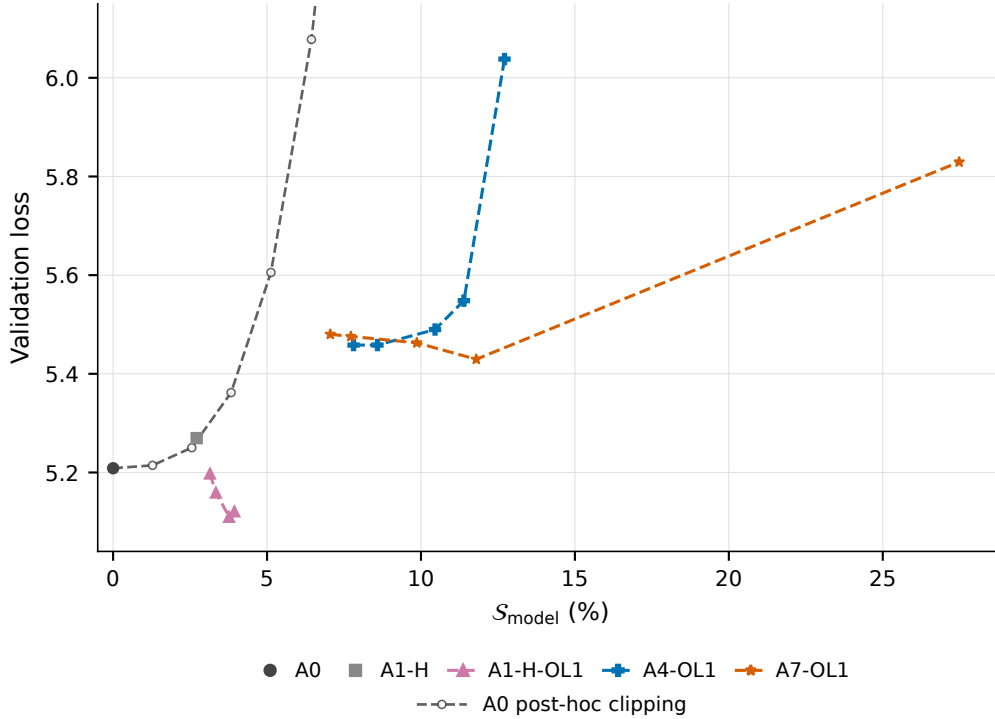


Figure 1: Quality-sparsity trade-offs from training interventions and post-hoc clipping. Validation loss versus model-wide logical sparsity $\mathcal{S}_{\text{model}}$ for Pythia-14M. Filled markers show 16 separately trained checkpoints: A0, A1-H, and the A1-H-OL1, A4-OL1 and A7-OL1 sweeps. Local OL1 varies $\lambda \in \{0.05, 0.1, 0.5, 1\}$; A4/A7-OL1 vary $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$ with fixed pressure weight $\lambda = 1$ and trust budget 1. The gray dashed path with open markers applies evaluation-only magnitude clipping at a, m, h, z to A0, with calibration targets $p = 0, 0.1, \dots, 0.9$. Six of its ten evaluations lie inside the displayed loss range; Appendix D.3 supplies the full-range view. Connections order evaluated settings and do not represent fitted Pareto envelopes or intermediate trained models. The same seed, initialization, data order and 712-update budget are used for the trained conditions. Loss and sparsity use all 338 complete 2,048-token blocks from 500 MiniPile validation documents, excluding the 1,444-token tail. Logical sparsity counts zero-operand products, not measured run-time savings.

tational reach: the A0 clipping reference acts at a, m, h, z , whereas A7 additionally gates the internal attention operands. At 14M, these site sets have analytic reach ceilings of 12.833% and 29.952%, respectively. The overview therefore motivates separating the effects of pressure and gate placement; a single ranking of complete recipes cannot explain their differences. Full endpoints and clipping trajectories are given in Appendix D.

4.2 Paired intervention effects

Pressure’s additional value depends on the gate setting and target set (Figure 2). Replacing GELU with ReLU alone increases model-wide sparsity by 2.714 percentage points (pp), but raises loss by 0.0611. Adding local pressure improves loss relative to this ReLU control. Changing naive L1 to OL1 lowers loss at three of the four matched weights, but reverses that improvement at $\lambda = 1$; the update rule is not uniformly better across the sweep.

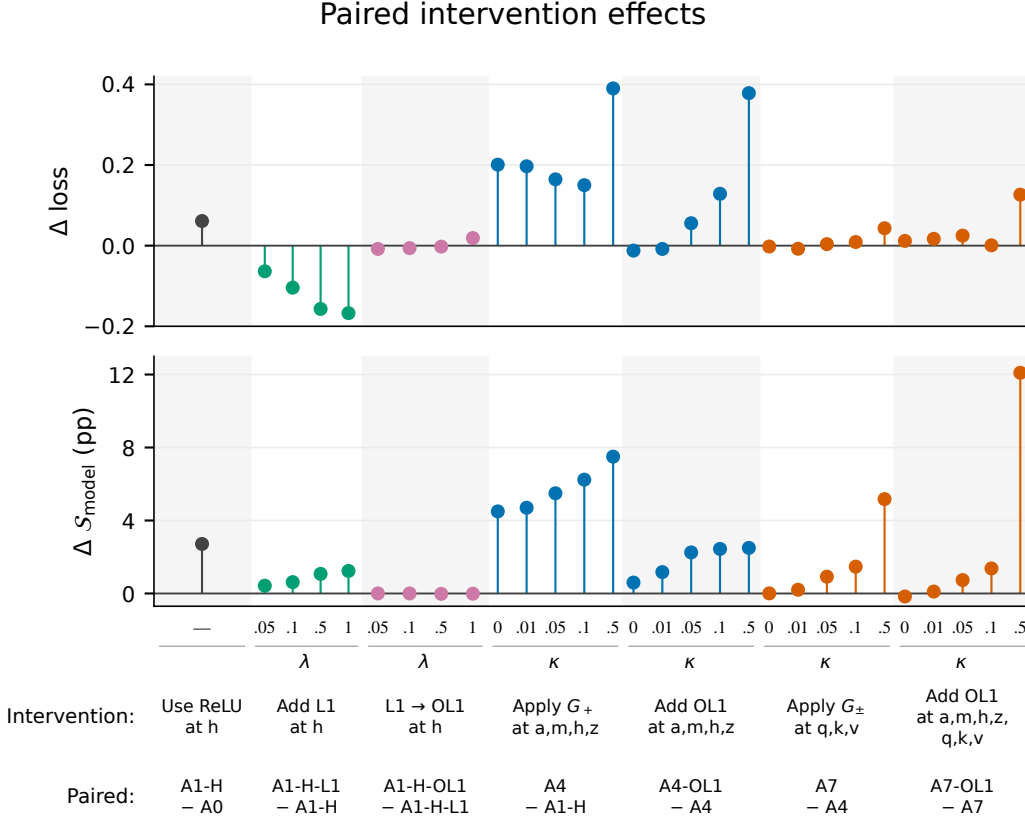


Figure 2: Paired intervention effects. Changes in validation loss (top; negative is better) and model-wide logical sparsity $\mathcal{S}_{\text{model}}$ (bottom; percentage points) for 29 comparisons of separately pretrained Pythia-14M conditions. Each delta is treatment minus reference, as specified in the **Paired:** row; zero denotes no change. The **Intervention:** row describes the operation and affected sites. Numeric labels give pressure weight λ or gate threshold κ ; the dash marks the ReLU replacement, which has no swept parameter. Parameter values increase from left to right with ordinal spacing. L1-to-OL1 comparisons share λ ; pressure additions to A4/A7 and the Q/K/V gate addition share κ . The remaining references are fixed controls: A0 uses GELU and A1-H uses ReLU. One-sided gates act at a,m,h,z ; symmetric Q/K gates follow RoPE. Four-site and seven-site orthogonal L1 (OL1) use fixed pressure weight $\lambda = 1$ and trust budget 1. Pairs share initialization, data order and training budget. All conditions use one seed and the full MiniPile validation set: 500 documents packed into 338 complete 2,048-token blocks, excluding the 1,444-token tail. These are descriptive paired comparisons, not successive training updates; logical sparsity does not measure runtime speedup.

For A4, adding OL1 improves both loss and sparsity at $\kappa = 0$ and 0.01. At larger thresholds it adds sparsity at increasing quality cost. At $\kappa = 0.5$, the increment is 2.498 pp for +0.3783 loss. The corresponding A7 pressure addition yields 12.096 pp for +0.1265 loss, while at $\kappa = 0.1$ it yields 1.372 pp for a much smaller +0.0008 loss change. Broader gates also contribute without pressure: A7 minus A4 at $\kappa = 0.5$ adds 5.171 pp for +0.0432 loss.

These contrasts establish that pressure’s response is conditional on the executed recipe. Comparing its response across A4 and A7 also changes the pressure objective and its normalization, so the difference cannot be assigned to gate placement alone. The next comparison examines how those complete recipes redistribute activation mass.

4.3 Reshaping activation distributions

The recipes produce different distributions even when both make FFN activations highly sparse (Figure 3). At $\kappa = 0.5$, A4-OL1 places 99.31% of pooled FFN elements exactly at zero, compared with 93.64% for A7-OL1. Their attention distributions differ more sharply: A4-OL1 has a narrow nonzero peak and only 0.22% exact zeros, whereas A7-OL1 has 95.60% exact zeros and nonzero density outside the symmetric threshold gap. The omitted zero masses are reported in Table 7; a narrow density peak is not itself a large exact-zero probability.

A4 does not directly gate or pressure q, k, v , yet their distributions change substantially. This is a nonlocal response under the complete A4-OL1 intervention, consistent with spillover. The comparison with A0 does not isolate pressure’s contribution from the other training changes. A7 directly intervenes at these attention sites, so its reshaping there has a different interpretation.

The operation counts show why this distinction matters model-wide. At the same threshold, QK and PV contribute 16.77 sparsity points under A7-OL1, versus 0.06 under A4-OL1 (Appendix D.2). Several projection contributions are smaller under A7-OL1, but its attention contribution more than offsets them. Greater model-wide sparsity can therefore coexist with less pooled FFN sparsity. This 14M diagnostic connects the observed distributions to counted work; the scale comparison tests the recipe ordering rather than replicating this distribution analysis.

4.4 Transfer across model sizes

The high-threshold A7-OL1 advantage persists at all three model sizes (Figure 4). At $\kappa = 0.5$, A7-OL1 has both lower loss and greater model-wide sparsity than A4-OL1 at 14M, 70M and 410M (Table 1). The zero-threshold ordering does not transfer unchanged: A4-OL1 is better on both axes at 14M and 70M, whereas A7-OL1 is better at 410M. Persistence concerns a specific recipe relationship, not the entire threshold response.

Raw sparsity must also be read against the changing workload. The A7 reach ceiling rises from 29.95% to 49.42% and 87.25% as the dense LM head accounts for a smaller share of counted products. The bottom row uses the same A7 denominator for every curve within a size: $U_{\text{arch}} = \mathcal{S}_{\text{model}} / \mathcal{S}_{\text{model}}^{\text{max}}(\text{A7})$. Because A7 reaches all counted block operations, this ratio equals the block zero-product fraction $\mathcal{S}_{\text{block}}$. It removes the dense-head share while retaining the different weights of operations within the blocks.

The larger cohorts compare complete A4-OL1 and A7-OL1 recipes; they lack the no-pressure A4/A7 controls needed to repeat the isolated pressure additions. They also share a token budget rather than tokens per parameter, and 410M uses a lower peak learning rate. The evidence thus supports one-seed, fixed-budget recipe transfer. It does not establish a scaling law or show that the distribution response has the same cause at every size.

Table 1: Complete-recipe contrasts at the boundary thresholds. Entries are A7-OL1 minus A4-OL1 at the same size and trained threshold. Negative loss changes and positive sparsity changes favor A7-OL1. These are matched one-seed point estimates over complete validation. Table 6 reports all five thresholds.

Size	κ	Δ loss	$\Delta \mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.5	-0.0703	+9.024

How interventions reshape activation distributions (Pythia-14M)

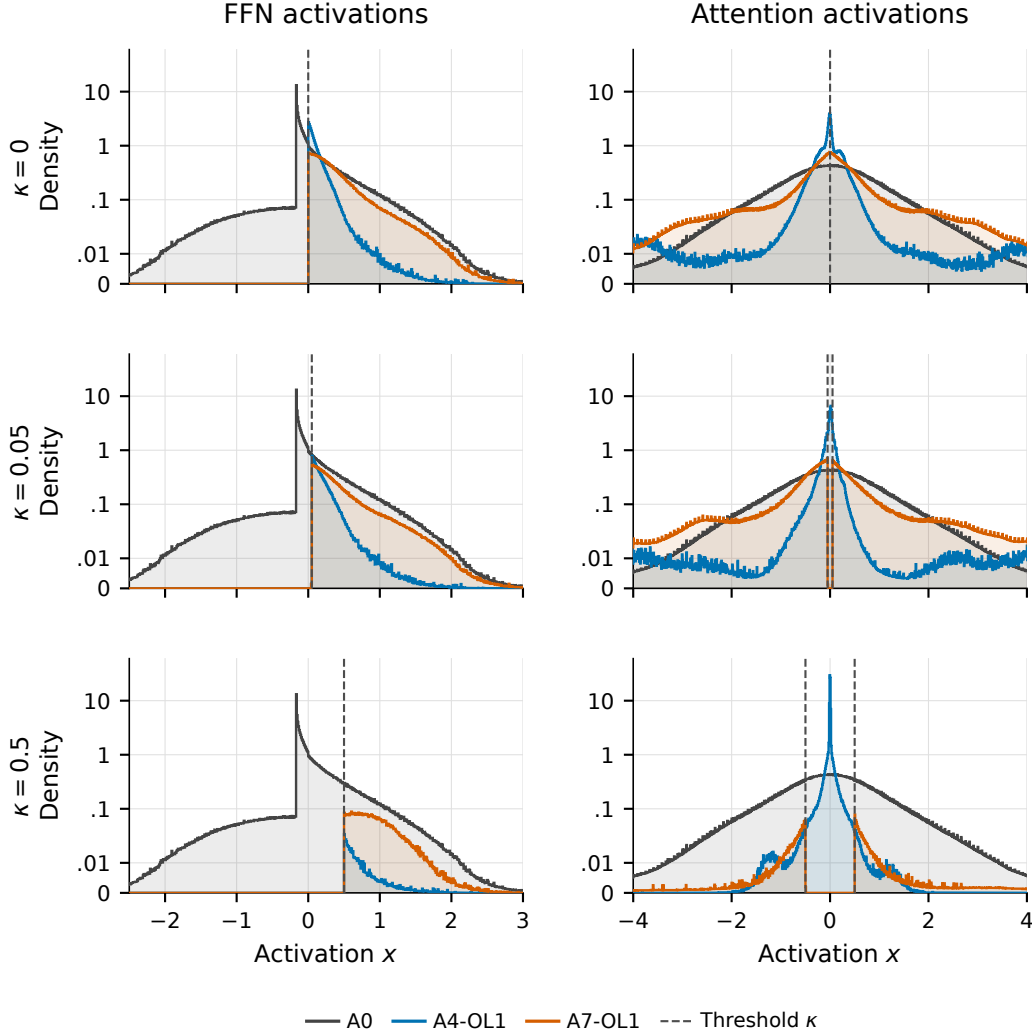


Figure 3: How interventions reshape activation distributions (Pythia-14M). Columns pool FFN activations (h, m) and attention activations (q, k, v) ; rows compare trained thresholds $\kappa = 0, 0.05, 0.5$. Gray, blue and orange lines with translucent fills show A0, A4-OL1 and A7-OL1. Integer histogram counts are pooled across all layers and validation blocks before division; FFN pooling weights $h : m$ by element counts (80:20), while q, k, v contribute equally. Signed activation x is linear; density uses a symlog scale, linear below 0.01. **Exact zeros are excluded from the curves and reported in Table 7.** Density divides nonzero bin counts by all captured elements and bin width; it is not renormalized to nonzero mass or the visible range. Dashed lines mark $+\kappa$ for A4/A7 FFN gates and $\pm\kappa$ for A7 attention gates, with Q/K measured after RoPE. A4 does not directly gate or pressure q, k, v ; A7 does both. The same A0 reference is repeated across rows. Seven frozen checkpoints each use all 338 complete blocks from 500 validation documents, excluding the 1,444-token tail. These are trained distributions with no additional post-hoc clipping. Complete histograms and tail counts accompany the figure; recipe differences do not isolate a causal pressure mechanism.

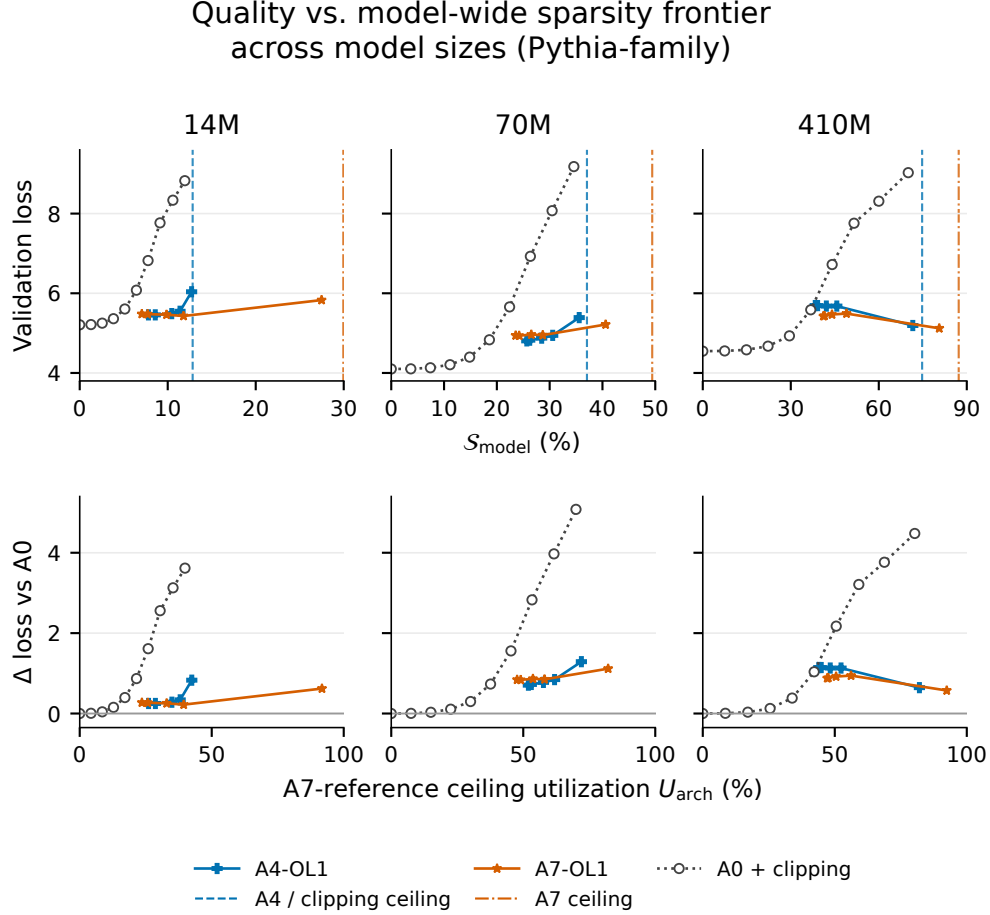


Figure 4: Quality-sparsity trade-offs across Pythia model sizes. Columns compare 14M, 70M and 410M. Solid curves connect separately trained A4-OL1 and A7-OL1 checkpoints at $\kappa = 0, 0.01, 0.05, 0.1, 0.5$; dotted open-marker curves show A0 post-hoc clipping at a, m, h, z for targets $p = 0, 0.1, \dots, 0.9$. The 60 evaluations appear in both rows. Top: validation loss versus $100\mathcal{S}_{\text{model}}$; vertical guides indicate A4/clipping-site and A7 analytic reach ceilings. Bottom: loss minus unmodified same-size A0 versus $100U_{\text{arch}}$, where $U_{\text{arch}} = \mathcal{S}_{\text{model}} / \mathcal{S}_{\text{model}}^{\text{max}}(\text{A7})$ for **every** curve. The A7 reference ceilings are 29.9524%, 49.4239% and 87.2452%; the A4 guide is not the normalization denominator. At $\kappa = 0.5$, A7-OL1 has lower loss and greater sparsity than A4-OL1 at all three sizes; the ordering at smaller thresholds is not uniform. Each size uses one seed, matched conditions and the same training-token budget, with a lower peak learning rate at 410M. Every evaluation covers 338 complete validation blocks from 500 documents, excluding the 1,444-token tail. This is transfer of a complete-recipe comparison, not an isolated pressure effect or a scaling law.

4.5 Realizing sparsity with specialized inference kernels

Logical sparsity becomes an execution benefit only if a kernel can skip work at a lower cost than performing it. We evaluate 42 proposals from a human-guided Codex search against native SDPA CUDA graphs on one RTX5090: BF16, batch-one, uncached 2,048-token inference with full vocabulary logits.

The best qualified incumbent reaches $1.7830\times$ on the fixed A7-OL1, $\kappa = 0.5$ search checkpoint (Figure 5, left). The selected kernel K050 qualifies on all 30 checkpoints in the 14M cohort, with geometric mean speedup $1.2340\times$. Speedup is positively associated with sparsity (right; OLS $R^2 = 0.8167$), but checkpoint quality, topology and weights vary; this is not an equal-quality acceleration comparison.

Relative to the same fusion with sparse paths disabled, K050 adds 31.1% acceleration at the search checkpoint and 4.32% geometrically across the cohort, helping 14 of 30 checkpoints. Attention skipping is unprofitable: disabling it improves every checkpoint, raising the mean to $1.2506\times$. Appendix D.4 details the implementation and ablations. Whether longer sequences amortize attention-skip overhead remains untested.

Specialized kernels thus realize part of the logical opportunity, with fusion and skip overhead determining the gain. Transfer to cached decoding, larger models or other hardware is not established.

Sparse kernel auto-research for full-model acceleration

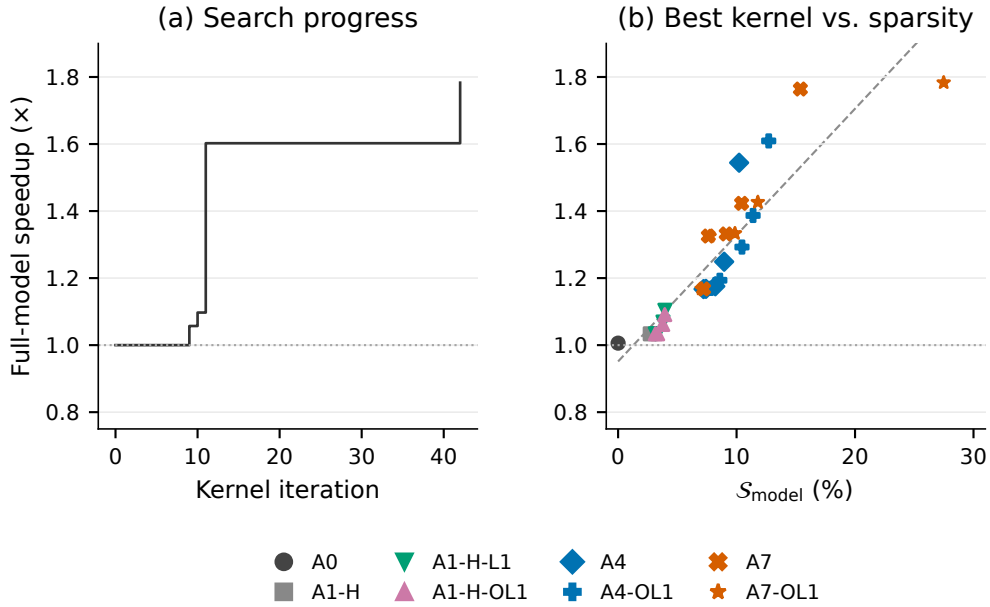


Figure 5: Sparse kernel auto-research for full-model acceleration. (a) Best qualified incumbent on fixed A7-OL1, $\kappa = 0.5$, across 42 iterations, starting at native $1\times$ and reaching $1.7830\times$. (b) K050 on 30 Pythia-14M checkpoints; symbols identify recipes. The dashed line is a descriptive OLS fit with intercept ($R^2 = 0.8167$); dotted lines mark native $1\times$. Speedup uses each checkpoint’s native SDPA CUDA-graph baseline on one RTX5090: BF16, batch one, uncached 2,048-token inference and full vocabulary logits. Qualification covers all 338 validation blocks. All checkpoints qualify, with geometric mean $1.2340\times$. Sparsity uses the canonical FP16 measurement; both speedup axes have restricted ranges. The association is not a causal sparsity effect. Appendix D.4 gives qualification details and ablations.

References

- Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2304.01373>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. *arXiv preprint arXiv:2402.02998*, 2024. URL <https://arxiv.org/abs/2402.02998>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. *arXiv preprint arXiv:2404.08763*, 2024. URL <https://arxiv.org/abs/2404.08763>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Yuqi Luo, Chenyang Song, Xu Han, Yingfa Chen, Chaojun Xiao, Xiaojun Meng, Liquan Deng, Jiansheng Wei, Zhiyuan Liu, and Maosong Sun. Sparsing Law: Towards Large Language Models with Greater Activation Sparsity. *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://proceedings.mlr.press/v267/luo25i.html>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Orel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv preprint arXiv:2502.10517*, 2025. URL <https://arxiv.org/abs/2502.10517>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A Intervention definitions

A.1 Gate semantics

For finite $\kappa \geq 0$, both threshold gates retain equality and leave retained values unchanged. The comparison mask is detached: the derivative with respect to the input is one for retained entries and zero for rejected entries. Consequently, $G_{+,0}$ and ReLU have identical forward values but differ at zero, where their derivatives are one and zero, respectively. $G_{\pm,0}$ is exactly the identity in both values and gradients. The gates use neither a straight-through estimator nor a top- k selection rule. Unselected sites retain their original transformations.

A.2 Activation pressure and OL1

For the J targeted site-layer tensors A_j , containing n_j entries each, the pressure objective is $\mathcal{L}_1 = J^{-1} \sum_{j=1}^J n_j^{-1} \sum_{i=1}^{n_j} |A_{j,i}|$. The tensors contain the executed site activations, after a gate where present. Absolute values and means are evaluated in float32. Each tensor receives equal weight regardless of its size; adding pressure sites changes both the objective and the weight of existing targets. Task and pressure losses are averaged over the same accumulation microbatches. For naive L1, the accumulated gradient of $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$ is globally norm-clipped before AdamW (Loshchilov and Hutter, 2019).

For OL1, let g_t and g_1 be the accumulated task and unweighted pressure gradients. We first clip g_t by its global norm and perform AdamW using only that clipped task gradient. Let $\hat{m}_t$ and $\hat{v}_t$ be its newly updated, bias-corrected moments. On parameters having both gradients and initialized optimizer state, define the adaptive task and pressure vectors $u = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon_A)$ and $w = g_1 / (\sqrt{\hat{v}_t} + \epsilon_A)$. The pressure gradient is unclipped and has no first-moment buffer. Inner products and norms below pool all these eligible parameters. With $c = \langle u, w \rangle$, $q = \|u\|_2^2$, and numerical stabilizer $\epsilon > 0$, the conflict-conditioned projection is

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (1)$$

For pressure weight $\lambda \geq 0$ and required budget $b > 0$, define the norm ratio $r = \lambda \|\tilde{w}\|_2 / (\|u\|_2 + \epsilon)$. Set $s = \min\{1, b/(r + \epsilon)\}$ when $r > 0$, or $s = 1$ otherwise. After AdamW, each eligible parameter group p , with learning rate α_p , receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$. The experiments' clipping thresholds, b , and λ are condition parameters.

This procedure is related to gradient-conflict projection (Yu et al., 2020) and the use of smoothed primary directions (Hsieh et al., 2024), but defines a particular correction in AdamW-preconditioned coordinates. The stabilized projection is only approximately orthogonal under conflict. Its geometry and budget precede multiplication by group learning rates and exclude decoupled weight decay. They therefore do not guarantee task-loss descent or preservation.

B Sparsity accounting

B.1 Exact counters

Local exact sparsity at site s is $\mathcal{Z}_s = \sum_{\ell,e,i} \mathbf{1}\{A_{s,i}^{(\ell,e)} = 0\} / \sum_{\ell,e} |A_s^{(\ell,e)}|$, where ℓ indexes layers, e evaluation batches, i tensor entries, and $|A|$ the number of elements in tensor A . Near-zero mass replaces equality by $|A_{s,i}| \leq \epsilon$ for a stated tolerance; it does not contribute to exact-zero counts unless the value is zero. All aggregate fractions pool integer numerators and denominators across batches and layers before division.

The atomic unit of model-wide accounting is one scalar multiplication in a matrix product. If $X \in \mathbb{R}^{n \times p}$ feeds a linear projection with q outputs, the count is $N = npq$, with $N_0 = q \sum_{i,j} \mathbf{1}\{X_{ij} = 0\}$. Zero weights receive no credit. For the multihead attention graph specified in Appendix C.1, both operands are activations, so a product is credited once if either is zero. Within one block and evaluation batch, for query position t , key position $u \leq t$, and within-head coordinate j ,

$$N_{0,QK} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{Q_{bhtj} = 0 \vee K_{bhuj} = 0\}, \quad (2)$$

$$N_{0,PV} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{P_{bhtu} = 0 \vee V_{bhuj} = 0\}. \quad (3)$$

Here P is the causal softmax probability tensor; the model numerator sums these counts over blocks and evaluation batches. Future masked positions are excluded from both counts, while zeros on valid positions, including floating-point underflow, are included. Counts use the operands actually consumed by each matrix product; upstream masks do not determine those operands' zeros after intervening transformations. Earlier value positions participate in more causal pairs, so local value sparsity alone does not determine PV sparsity.

B.2 Reach accounting and normalization

For a specified graph and workload, let N_o count the valid scalar multiplications in block operation family o , pooled over layers and examples. Then $N_{\text{model}} = \sum_o N_o + N_{\text{head}}$, while N_0 sums the corresponding zero-operand counts only in block operations. The block-only view is $\mathcal{S}_{\text{block}} = N_0 / \sum_o N_o$. Both sparsities are fractions; percentages multiply them by 100. Embedding lookup, normalization, nonlinearities, softmax, RoPE, bias and residual additions, and data movement are outside this accounting.

To construct the reach numerator, count products guaranteed to have a zero activation operand as a structural consequence of setting the selected sites to zero, independently of checkpoint values or natural zeros elsewhere. Include exact zero-preserving propagation under the declared graph rules. Products reached through more than one site receive one credit. The resulting count is $N_{\text{reach}}(\mathcal{G})$. Division by the same N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$. Natural zeros outside this selected-site reach can contribute to observed $\mathcal{S}_{\text{model}}$, so the ceiling is not an upper bound on every observation. Its gap to an observation does not measure sparsity available without quality loss. Appendix C.2 specializes the denominator and reach rules to the Pythia graph used in the experiments.

C Experimental setup details

C.1 Pythia sites and gate placement

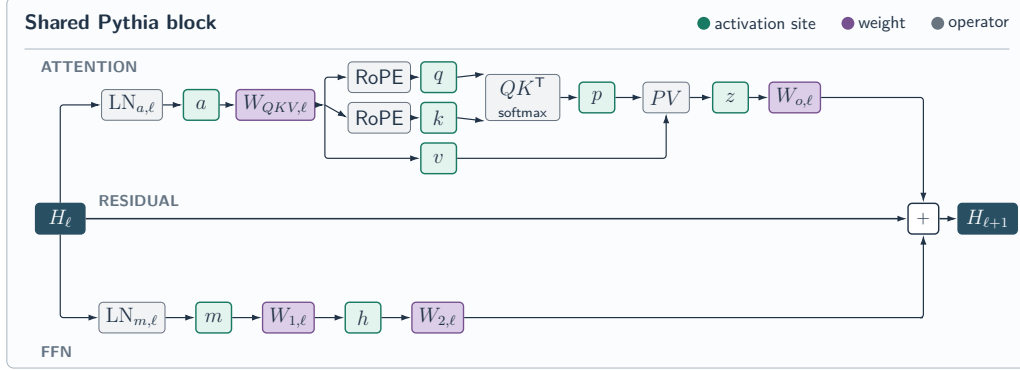
Pythia uses parallel attention and FFN branches with separate LayerNorms. For batch size B , sequence length T , residual width d , FFN width d_f , and H heads, let $d_h = d/H$. Table 2 identifies the actual operands recorded after any selected gate. The same site mapping is applied in every layer, with one common threshold per condition. The recipe and pressure assignments are defined in Section 4.

At an ungated h , the activation is GELU; a selected gate replaces GELU. At other ungated sites the map is the identity applied to that port's input. At zero threshold, A7's additional symmetric query/key/value gates are exact identities in both values and gradients. They therefore do not change A4's mathematical forward map in the absence of pressure. A7-OL1 still differs from A4-OL1 in its pressure targets and their equal-tensor normalization.

Query and key counts use actual post-RoPE operands: rotation can repopulate an individual zero coordinate. Likewise, the output projection uses actual post-gate context z , rather than inferring its zeros from the value mask.

Table 2: Intervention sites and the matrix products that consume them directly.

Site	Shape	Consuming operation	Placement
a	$B \times T \times d$	QKV projection	After attention LayerNorm
m	$B \times T \times d$	FFN up projection	After FFN LayerNorm
h	$B \times T \times d_f$	FFN down projection	Nonlinearity output
q, k	$B \times H \times T \times d_h$	Attention scores	After partial RoPE
v	$B \times H \times T \times d_h$	Probability–value	After the QKV split
z	$B \times T \times d$	Attention output	Concatenated context, before output projection



Sparsification ladder

Non-linearity ● GeLU ● ReLU ● $G_{+, \kappa}$ ● $G_{\pm, \kappa}$ ● none

Step	Labels	Intervention								$\mathcal{S}_{\text{model}}^{\text{max}}$ at $T = 2,048$		
		h	m	a	z	q	k	v	Pressure	14M	70M	410M
1	A0	●	●	●	●	●	●	●	none	0.0%	0.0%	0.0%
2	A1-H	●	●	●	●	●	●	●	none	4.3%	12.4%	24.9%
3	A1-H-L1	●	●	●	●	●	●	●	L1N	4.3%	12.4%	24.9%
4	A1-H-OL1	●	●	●	●	●	●	●	OL1	4.3%	12.4%	24.9%
5	A4	●	●	●	●	●	●	●	none	12.8%	37.1%	74.8%
6	A4-OL1	●	●	●	●	●	●	●	OL1	12.8%	37.1%	74.8%
7	A7	●	●	●	●	●	●	●	none	30.0%	49.4%	87.2%
8	A7-OL1	●	●	●	●	●	●	●	OL1	30.0%	49.4%	87.2%

Figure 6: Shared Pythia block and sparsification intervention ladder. The map separates gate placement, gate family, and pressure; L1N denotes naive L1. The final columns show analytic selected-site reach $100 \cdot \mathcal{S}_{\text{model}}^{\text{max}} \%$ at $T = 2,048$, not observed sparsity or speedup. Pressure targets all gated sites in each pressure-bearing row. Rows define candidate recipes; their experimental coverage is reported with the results.

C.2 Architecture-specific counts and reach

The operation inventory comprises QKV projection, attention scores (QK), probability–value products (PV), attention-output projection, and the two FFN projections. For one uncached, unpadded sequence, each block contributes $3Td^2$ products from QKV projection, Td^2 from the attention-output projection, Tdd_f from each FFN projection, and $dT(T+1)/2$ from each of QK and PV .

Thus, for L blocks and vocabulary size V_{vocab} ,

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T + 1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \tag{4}$$

Over B sequences, $N_{\text{model}} = BC_{\text{model}}$; the dense head contributes only to the denominator, as defined in Section 3.2.

Let $\mathcal{O}(\mathcal{G})$ be the union of operations reachable when the selected sites are all zero: a reaches QKV; m reaches FFN up; h reaches FFN down; either q or k reaches QK; v reaches PV and attention output; and z reaches attention output. Each operation is counted once. The v rule includes the exact closure $V = 0 \Rightarrow PV = 0$. No further propagation is assumed: QKV biases can survive $a = 0$, and zero scores do not imply zero softmax probabilities. For the same site set in every layer, $N_{\text{reach}} = BL \sum_{o \in \mathcal{O}(\mathcal{G})} C_o$, where C_o is that operation’s per-sequence count. Division by N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$.

With Pythia’s $d_f = 4d$, the per-block reachable counts for A0, A1-H, A4, and A7 are 0, $4Td^2$, $12Td^2$, and C_{block} , respectively. The ladder evaluates these formulas at $T = 2,048$, $V_{\text{vocab}} = 50,304$, and the pinned (L, d) configurations (6, 128), (6, 512), and (24, 1,024).¹ No checkpoint enters this calculation. In particular, A0 has zero selected-site reach but may have positive observed sparsity from natural zeros. The ceiling is consequently not an upper bound on every observed $\mathcal{S}_{\text{model}}$, nor does its gap to an observation measure sparsity available without quality loss.

C.3 Evaluation scope

Quality and logical counts are paired in the same eager-attention evaluation of a checkpoint, with its declared gates and evaluation precision. The complete MiniPile validation split contains 500 documents. Concatenation and packing produce 338 complete blocks of 2,048 tokens (692,224 input tokens); the final 1,444-token tail is excluded. Each block contributes 2,047 next-token targets to cross-entropy, while product accounting includes all 2,048 input positions. Loss is averaged over the equally sized blocks, and sparsity is count-pooled. This workload uses no key–value cache. Condition-specific training budgets, optimizer settings, seeds, and checkpoint selection accompany their results; they are not implied by the ladder’s list of candidate conditions.

C.4 Realized training protocol

All conditions start from random weights drawn from the pinned Pythia architecture configurations; released pretrained weights are not used. Model initialization and data-order seeds are both 1234. Within a size, the conditions share the initial parameter hash and training schedule. The 30-condition 14M cohort contains A0, A1-H, four weights each for A1-H-L1 and A1-H-OL1, and five thresholds each for A4, A4-OL1, A7 and A7-OL1. Each larger cohort contains A0, A1-H, and the five-threshold A4-OL1/A7-OL1 sweeps. Local pressure weights are $\{0.05, 0.1, 0.5, 1\}$; gate thresholds are $\{0, 0.01, 0.05, 0.1, 0.5\}$. Multisite OL1 uses $\lambda = 1$, with pressure at all four or seven selected sites. Every OL1 condition uses trust budget 1.

Training uses AdamW with $\beta = (0.9, 0.95)$, $\varepsilon = 10^{-8}$, weight decay 0.1 and global task-gradient clipping at norm 1. Naive L1 clips the combined objective gradient instead. The learning rate warms up linearly for eight updates and follows cosine decay to one tenth of its peak. Parameters and optimizer state are FP32; forward computation uses FP16 autocast with dynamic loss scaling. Attention and hidden dropout are zero. Table 3 gives size-dependent settings. The global batch is 1,024 sequences of 2,048 tokens, for 712 optimizer updates and 1,493,172,224 input tokens. This wraps 714 blocks beyond the 728,374 complete blocks in one packed training pass. We use the final checkpoint without validation-based checkpoint selection. The lower 410M learning rate is part of the realized protocol, so size alone is not the only cross-scale change.

¹Architecture configurations: EleutherAI’s Pythia-14M-deduped, Pythia-70M-deduped, and Pythia-410M-deduped. Links identify immutable configuration revisions; released weights are not used.

Table 3: Size-dependent training settings. Tokens per parameter use the common realized input-token budget; they are not held fixed across sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation	32	256	256
Tokens per parameter	106.14	21.20	3.68

D Complete results and supporting diagnostics

The following tables report every selected trained endpoint, all 29 paired 14M contrasts and all 15 cross-scale recipe comparisons. They retain dominated settings as well as favorable ones. Each entry summarizes one trained model on complete validation; the tables do not estimate variation over training seeds. Sparsity percentages are formed from pooled integer product counts. The endpoint table uses the common A7 reach denominator from Figure 4, rather than a different denominator for each recipe.

D.1 Exact-zero mass and signed-density coverage

The seven frozen-checkpoint evaluations underlying Figure 3 record signed values after the trained gates, with Q/K measured after RoPE. Each checkpoint contributes 2,658,140,160 FFN elements and 1,594,884,096 attention elements across the complete validation set. Summing integer counts gives the 80:20 weighting of $h : m$ and equal weighting of $q : k : v$. These pooled distributions are element-weighted mixtures, not distributions of layer means.

Exact zeros are stored separately from the nonzero histogram. The native grid spans $[-8, 8]$ with 16,000 bins of nominal width 0.001; the figure combines ten bins at a time and divides by the actual floating-point bin width and the count of all captured elements. Consequently, a curve’s full integral is the recorded nonzero mass inside that grid, not one. Table 7 reports the zero mass and the fractions outside the displayed and native ranges. Underflow and overflow counts are retained rather than folded into edge bins. The displayed limits are $[-2.5, 3]$ for FFN and $[-4, 4]$ for attention. The largest omitted display tail is 6.6719%; the native-grid tail is at most 0.7607%.

The diagnostic reruns use FP32 parameters with FP16 autocast and no additional clipping. Their maximum absolute validation-loss difference from the retained reference is 6.89×10^{-5} , below the specified 5×10^{-4} tolerance; all 108 trained-gate checks pass. Small rounding differences in exact-zero counts between evaluation passes do not change the displayed conclusions. Complete per-site and pooled histograms, counts and checkpoint hashes accompany the data release.

Table 4: All 54 trained endpoints. λ is pressure weight; κ is the trained gate threshold. $U_{A7} = \mathcal{S}_{\text{model}}/\mathcal{S}_{\text{model}}^{\max}(\text{A7})$ uses the same A7 reference within each model size, including for A0. Rounded zero sparsity need not be exactly zero.

Size	Recipe	Parameter	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{A7} (%)
14M	A0	—	5.2086	0.000	0.000
14M	A1-H	—	5.2696	2.714	9.061
14M	A1-H-L1	$\lambda = 0.05$	5.2061	3.142	10.489
14M	A1-H-L1	$\lambda = 0.1$	5.1655	3.336	11.139
14M	A1-H-L1	$\lambda = 0.5$	5.1127	3.788	12.645
14M	A1-H-L1	$\lambda = 1$	5.1023	3.949	13.185
14M	A1-H-OL1	$\lambda = 0.05$	5.1981	3.145	10.501
14M	A1-H-OL1	$\lambda = 0.1$	5.1594	3.339	11.146
14M	A1-H-OL1	$\lambda = 0.5$	5.1102	3.768	12.580
14M	A1-H-OL1	$\lambda = 1$	5.1212	3.938	13.149
14M	A4	$\kappa = 0$	5.4705	7.212	24.078
14M	A4	$\kappa = 0.01$	5.4665	7.414	24.752
14M	A4	$\kappa = 0.05$	5.4341	8.206	27.396
14M	A4	$\kappa = 0.1$	5.4197	8.953	29.891
14M	A4	$\kappa = 0.5$	5.6597	10.216	34.106
14M	A4-OL1	$\kappa = 0$	5.4583	7.810	26.075
14M	A4-OL1	$\kappa = 0.01$	5.4583	8.587	28.668
14M	A4-OL1	$\kappa = 0.05$	5.4898	10.456	34.910
14M	A4-OL1	$\kappa = 0.1$	5.5483	11.394	38.041
14M	A4-OL1	$\kappa = 0.5$	6.0380	12.713	42.446
14M	A7	$\kappa = 0$	5.4684	7.218	24.097
14M	A7	$\kappa = 0.01$	5.4588	7.618	25.433
14M	A7	$\kappa = 0.05$	5.4379	9.127	30.471
14M	A7	$\kappa = 0.1$	5.4287	10.425	34.805
14M	A7	$\kappa = 0.5$	5.7029	15.387	51.371
14M	A7-OL1	$\kappa = 0$	5.4802	7.054	23.551
14M	A7-OL1	$\kappa = 0.01$	5.4758	7.723	25.786
14M	A7-OL1	$\kappa = 0.05$	5.4628	9.863	32.929
14M	A7-OL1	$\kappa = 0.1$	5.4295	11.797	39.385
14M	A7-OL1	$\kappa = 0.5$	5.8294	27.483	91.755
70M	A0	—	4.0998	0.000	0.001
70M	A1-H	—	4.2227	10.066	20.366
70M	A4-OL1	$\kappa = 0$	4.8054	25.673	51.944
70M	A4-OL1	$\kappa = 0.01$	4.8229	26.224	53.060
70M	A4-OL1	$\kappa = 0.05$	4.8742	28.490	57.644
70M	A4-OL1	$\kappa = 0.1$	4.9439	30.574	61.861
70M	A4-OL1	$\kappa = 0.5$	5.3895	35.596	72.022
70M	A7-OL1	$\kappa = 0$	4.9412	23.562	47.674
70M	A7-OL1	$\kappa = 0.01$	4.9364	24.226	49.017
70M	A7-OL1	$\kappa = 0.05$	4.9637	26.524	53.666
70M	A7-OL1	$\kappa = 0.1$	4.9501	28.694	58.056
70M	A7-OL1	$\kappa = 0.5$	5.2159	40.602	82.150
410M	A0	—	4.5475	0.011	0.013
410M	A1-H	—	4.6513	21.050	24.127
410M	A4-OL1	$\kappa = 0$	5.6921	38.339	43.944
410M	A4-OL1	$\kappa = 0.01$	5.6914	39.149	44.873
410M	A4-OL1	$\kappa = 0.05$	5.6789	42.125	48.283
410M	A4-OL1	$\kappa = 0.1$	5.6789	45.688	52.367
410M	A4-OL1	$\kappa = 0.5$	5.1910	71.591	82.058
410M	A7-OL1	$\kappa = 0$	5.4263	41.121	47.133
410M	A7-OL1	$\kappa = 0.01$	5.4331	41.478	47.542
410M	A7-OL1	$\kappa = 0.05$	5.4679	44.053	50.494
410M	A7-OL1	$\kappa = 0.1$	5.4907	48.998	56.161
410M	A7-OL1	$\kappa = 0.5$	5.1207	80.616	92.401

Table 5: All 29 Pythia-14M contrasts, treatment minus reference. When both recipes have a swept parameter, its value is matched; A0 and A1-H are fixed controls.

Treatment – reference	Parameter	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
A1-H – A0	—	+0.0611	+2.714
A1-H-L1 – A1-H	$\lambda = 0.05$	-0.0635	+0.427
A1-H-L1 – A1-H	$\lambda = 0.1$	-0.1042	+0.622
A1-H-L1 – A1-H	$\lambda = 0.5$	-0.1569	+1.073
A1-H-L1 – A1-H	$\lambda = 1$	-0.1674	+1.235
A1-H-OL1 – A1-H-L1	$\lambda = 0.05$	-0.0081	+0.004
A1-H-OL1 – A1-H-L1	$\lambda = 0.1$	-0.0061	+0.002
A1-H-OL1 – A1-H-L1	$\lambda = 0.5$	-0.0025	-0.020
A1-H-OL1 – A1-H-L1	$\lambda = 1$	+0.0189	-0.011
A4 – A1-H	$\kappa = 0$	+0.2009	+4.498
A4 – A1-H	$\kappa = 0.01$	+0.1969	+4.700
A4 – A1-H	$\kappa = 0.05$	+0.1645	+5.492
A4 – A1-H	$\kappa = 0.1$	+0.1500	+6.239
A4 – A1-H	$\kappa = 0.5$	+0.3900	+7.501
A4-OL1 – A4	$\kappa = 0$	-0.0122	+0.598
A4-OL1 – A4	$\kappa = 0.01$	-0.0082	+1.173
A4-OL1 – A4	$\kappa = 0.05$	+0.0556	+2.251
A4-OL1 – A4	$\kappa = 0.1$	+0.1286	+2.441
A4-OL1 – A4	$\kappa = 0.5$	+0.3783	+2.498
A7 – A4	$\kappa = 0$	-0.0021	+0.006
A7 – A4	$\kappa = 0.01$	-0.0077	+0.204
A7 – A4	$\kappa = 0.05$	+0.0038	+0.921
A7 – A4	$\kappa = 0.1$	+0.0090	+1.472
A7 – A4	$\kappa = 0.5$	+0.0432	+5.171
A7-OL1 – A7	$\kappa = 0$	+0.0118	-0.163
A7-OL1 – A7	$\kappa = 0.01$	+0.0170	+0.106
A7-OL1 – A7	$\kappa = 0.05$	+0.0249	+0.736
A7-OL1 – A7	$\kappa = 0.1$	+0.0008	+1.372
A7-OL1 – A7	$\kappa = 0.5$	+0.1265	+12.096

Table 6: All matched A7-OL1 minus A4-OL1 comparisons. Negative loss changes and positive sparsity changes favor A7-OL1.

Size	κ	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.01	+0.0175	-0.863
14M	0.05	-0.0270	-0.593
14M	0.1	-0.1188	+0.402
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.01	+0.1135	-1.998
70M	0.05	+0.0895	-1.966
70M	0.1	+0.0062	-1.880
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.01	-0.2582	+2.329
410M	0.05	-0.2111	+1.929
410M	0.1	-0.1881	+3.310
410M	0.5	-0.0703	+9.024

Table 7: Zero mass omitted from the density curves, and mass outside their displayed range (view tail) and the native $[-8, 8]$ histogram (grid tail). All percentages divide integer counts by all captured elements. A0 is listed once although it is repeated in each figure row. These 14 groups come from seven checkpoints.

Recipe	κ	Group	Zero (%)	View tail (%)	Grid tail (%)
A0	—	FFN	< 0.01	0.1113	0.0000
A0	—	Attention	< 0.01	2.3929	0.4632
A4-OL1	0	FFN	67.22	0.0003	0.0000
A4-OL1	0	Attention	< 0.01	5.1717	0.6162
A4-OL1	0.05	FFN	86.71	0.0003	0.0000
A4-OL1	0.05	Attention	< 0.01	5.6601	0.7607
A4-OL1	0.5	FFN	99.31	0.0001	0.0000
A4-OL1	0.5	Attention	0.22	0.0003	0.0000
A7-OL1	0	FFN	64.52	0.0063	0.0000
A7-OL1	0	Attention	< 0.01	5.4657	0.4648
A7-OL1	0.05	FFN	75.29	0.0106	0.0000
A7-OL1	0.05	Attention	7.26	6.6719	0.6932
A7-OL1	0.5	FFN	93.64	0.0037	0.0000
A7-OL1	0.5	Attention	95.60	0.2396	0.0009

D.2 Operation contributions to model-wide sparsity

An operation’s contribution to $\mathcal{S}_{\text{model}}$ is its zero-product count divided by the full model product count. Summing these contributions recovers the model-wide fraction; summing local activation percentages would not. Table 8 gives this decomposition at $\kappa = 0.5$. At 14M, extending from A4-OL1 to A7-OL1 adds attention opportunity while reducing the contribution from some projections. At larger sizes, projection work carries more weight in the workload, so an equal change in a local zero fraction need not yield an equal change in model-wide sparsity. Figure 7 displays the operation-level view at the same threshold and all three sizes.

Table 8: Contributions to $100\mathcal{S}_{\text{model}}$ in percentage points at $\kappa = 0.5$. All six operations share the same model denominator within each size. The dense LM head contributes to that denominator but receives no zero credit. Differences in rounded entries can affect the displayed sum.

Operation	14M		70M		410M	
	A4-OL1	A7-OL1	A4-OL1	A7-OL1	A4-OL1	A7-OL1
QKV	3.173	2.430	8.820	6.231	17.563	16.992
<i>QK</i>	0.000	8.324	0.000	5.052	0.000	5.206
<i>PV</i>	0.057	8.449	0.011	5.371	0.008	5.858
Attention output	1.069	1.069	3.087	3.084	6.186	6.187
FFN up	4.139	2.939	11.329	8.526	22.938	21.503
FFN down	4.275	4.272	12.350	12.338	24.895	24.869
Total	12.713	27.483	35.596	40.602	71.591	80.616

Operation contributions at $\kappa = .5$

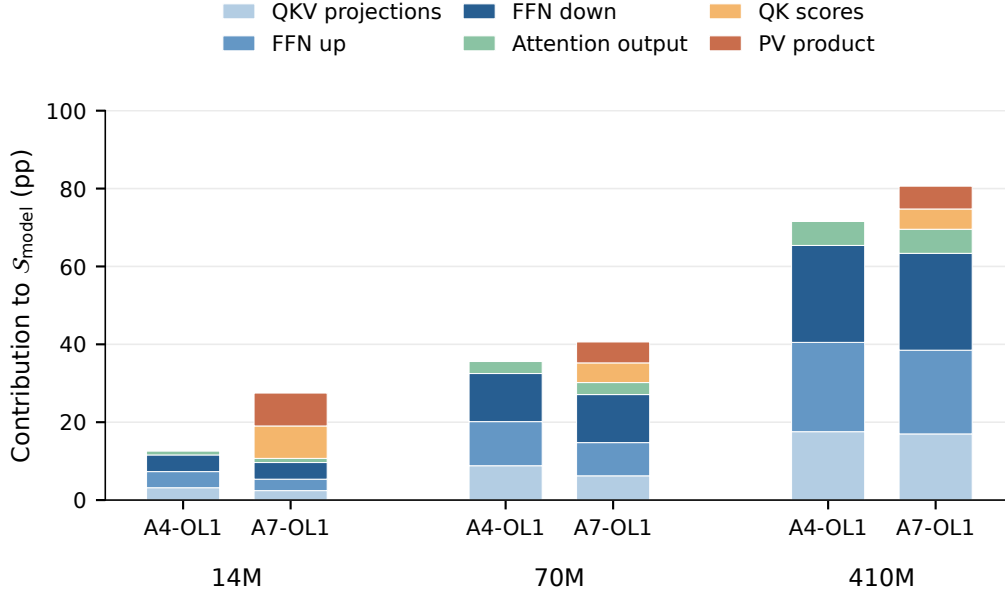


Figure 7: Operation accounting for A4-OL1 and A7-OL1 at $\kappa = 0.5$ across 14M, 70M and 410M. Zero-operand products are counted over causally valid positions and pooled across all 338 validation blocks. Each contribution uses the full model denominator, including the dense LM head without zero credit. The decomposition distinguishes the amount of work affected at a site from its local zero fraction. These are logical multiplication opportunities, not executed instruction counts or measured speedups.

D.3 Complete post-hoc clipping trajectories

The complete release contains 540 evaluations: ten clipping targets for each of the 54 trained checkpoints. It joins 350 new evaluations with 190 retained evaluations matched by checkpoint content hash. Calibration uses the first ten complete training blocks in source order, separately for each layer’s a, m, h, z activations. Evaluation zeros values satisfying $|x| \leq t$ after the trained gates, without changing weights or A7’s internal attention gates. The target p controls a calibration quantile; it is not the achieved model-wide sparsity. In particular, an existing zero mass can make several targets identical.

Figures 8–10 retain the full loss range and every target, including dominated points and ties. Each path begins at its actually measured $p = 0$ evaluation. Small numerical differences from the canonical training endpoint can change strict nondominance without implying a substantive improvement. Numerical frontier memberships are stored separately from the lines connecting evaluated settings. All evaluations use complete validation with FP32 parameters, FP16 autocast, eager attention, batch one and $T = 2,048$.

14M training and post-hoc clipping

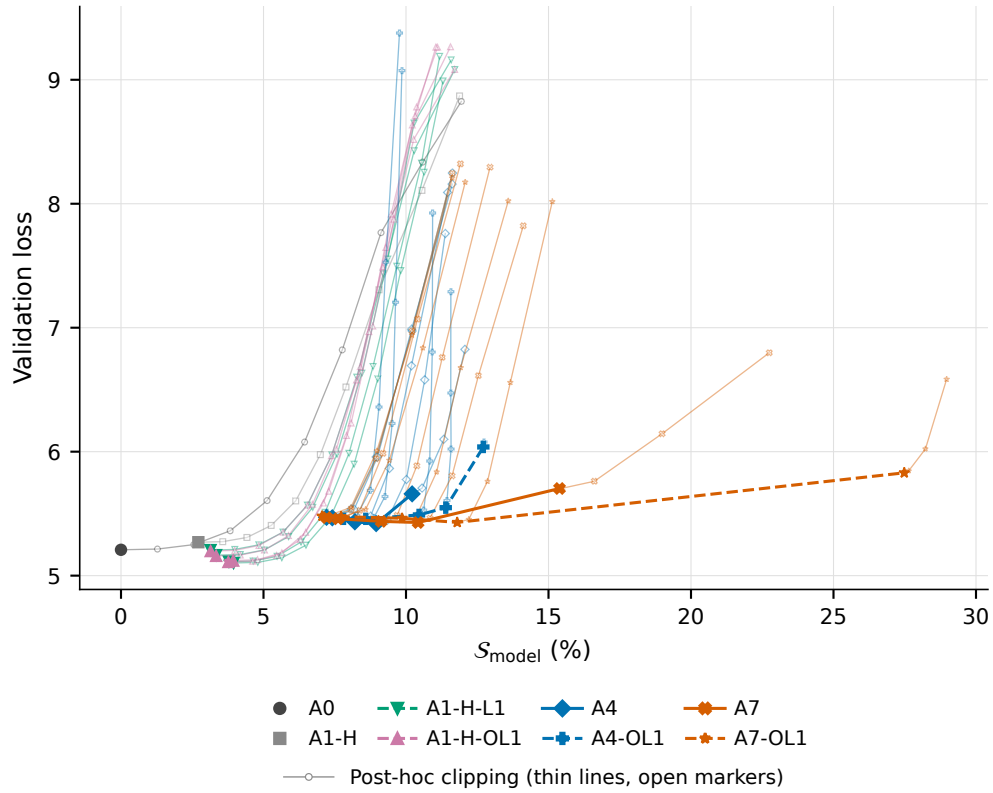


Figure 8: Complete 14M training and post-hoc clipping results: 30 trained endpoints and 300 clipping evaluations. Large filled markers denote trained endpoints; bold lines connect their settings, solid for unpressured recipes and dashed for L1/OL1. Each faint open-marker path follows one fixed checkpoint through $p = 0, 0.1, \dots, 0.9$. Colors and symbols identify its source recipe. The full loss range is shown. Connections order evaluated settings and do not interpolate attainable models or fitted Pareto envelopes.

70M training and post-hoc clipping

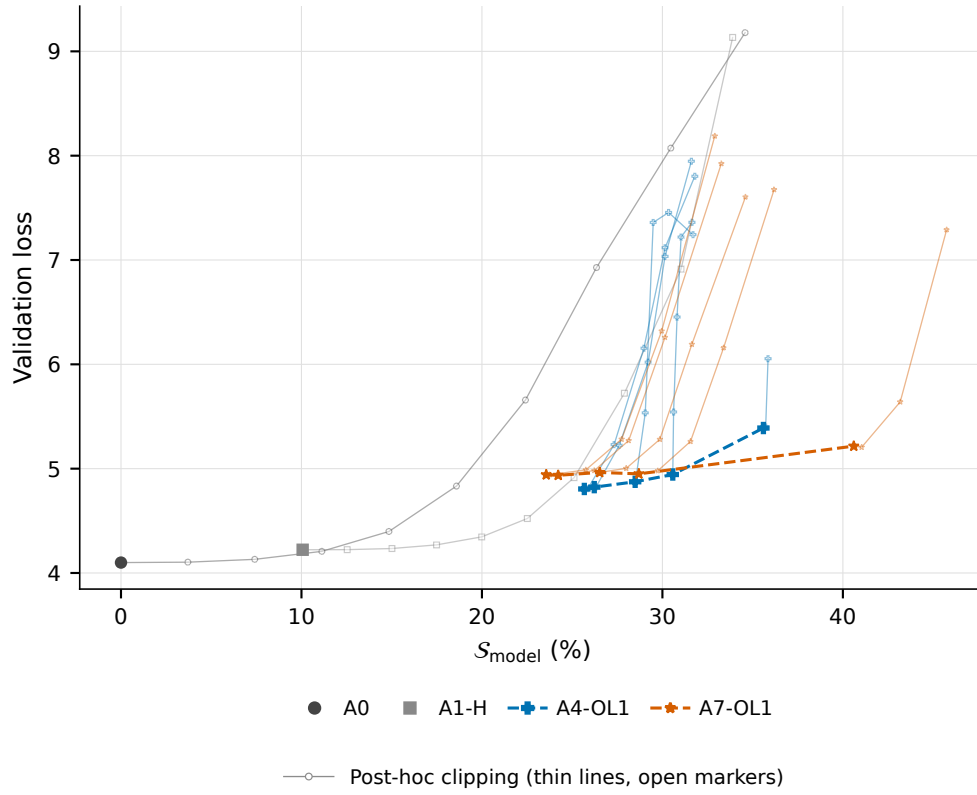


Figure 9: Complete 70M training and post-hoc clipping results: 12 trained endpoints and 120 clipping evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing clipping targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 8.

410M training and post-hoc clipping

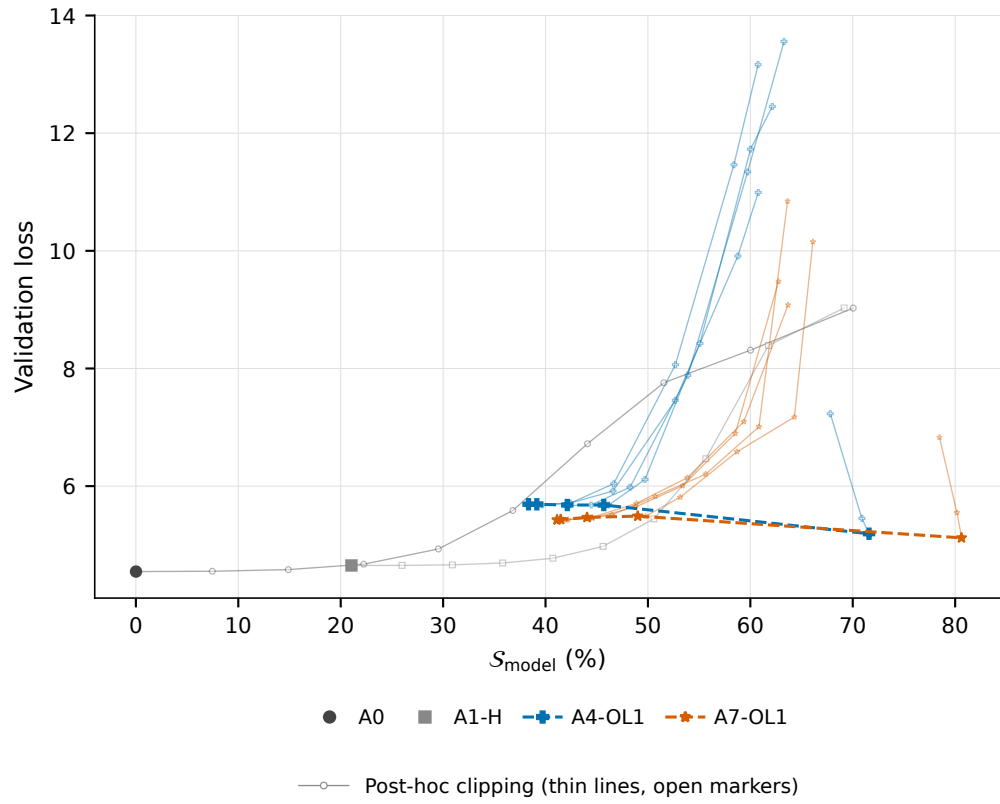


Figure 10: Complete 410M training and post-hoc clipping results: 12 trained endpoints and 120 clipping evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing clipping targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 8.

D.4 Kernel qualification and ablations

The retrospective replays frozen proposals on the same 30-checkpoint 14M cohort used throughout the manuscript. The historical five-checkpoint A4 cohort with pressure only at h is excluded. Search progress and the final sweep are separate measurements, explaining the small difference between their $1.7830\times$ and $1.7832\times$ maxima.

The coding-agent configuration records *gpt-6-astra* as the default model, not the model served for every request. This is one human-guided development trajectory and does not compare agent-assisted with human-only kernel development.

Every candidate must produce finite outputs, with each logit satisfying $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$, per-block relative output L2 error at most 0.02, and absolute pooled loss difference at most 0.001. Here $\hat{y}$ is the unmodified native eager reference. All three fresh processes must pass on all 338 validation blocks. Qualification is bounded numerical agreement on these inputs, not bitwise equivalence. Failed candidates are excluded from qualified speedup summaries; P0 qualifies on only four checkpoints, so its average is not comparable to a 30-checkpoint cohort average.

Timing uses the same checkpoint, inputs and CUDA-graph scaffold for native PyTorch/SDPA and each candidate. For each checkpoint, we take the geometric mean of 1,344 paired host-latency ratios (native divided by candidate): 64 inputs, seven randomized paired passes, and three fresh processes. Compilation, static weight preparation, graph capture and equal input staging are excluded. Preprocessing that depends on activations, embeddings, every block, final normalization and full vocabulary logits are included. There is no activation cache. Cohort summaries give each checkpoint equal weight.

K050 combines normalization and RoPE fusion, zero-fragment tensor-core bypass and sparse-row projection paths, using CUDA, CUTLASS, FlashAttention and PyTorch components. The all-skips-off control retains fusion while disabling both attention and projection skips; the attention-dense control retains projection skips. Their ratios are formed from separately measured, native-normalized timings. K050’s sparse paths improve 14 of 30 checkpoints relative to all-skips-off, with geometric mean ratio $1.0432\times$. Disabling attention skipping improves all 30: at the search checkpoint, instrumentation confirms that skipping bypasses 58.0% of eligible *QK* and 67.2% of *PV* MMA work, yet its overhead exceeds the saved time. Scalar zero-product fractions and skipped MMA fractions are different estimands.

Table 9: Numerically qualified full-model speedups against native SDPA CUDA graphs. GM is the equal-checkpoint geometric mean; the range is across qualified checkpoints, not a confidence interval. All conditions use one RTX5090, BF16, batch one, 2,048 tokens and full vocabulary logits. P0’s four-checkpoint subset is shown only to document its limited qualification.

Implementation	Qualified	GM speedup	Range
K049	30/30	1.1141	0.9550–1.5284
K050	30/30	1.2340	1.0059–1.7832
K050, attention dense	30/30	1.2506	1.0185–1.8022
K050, all skips off	30/30	1.1829	1.0503–1.3628
P0 (qualified subset only)	4/30	0.8910	0.8180–1.0012

D.5 Evidence and data accompanying the draft

The accompanying data directory contains all 54 training records with integer operation counts, all 540 clipping evaluations and frontier memberships, seven signed histograms with zero and tail counts, figure coordinates, and the 30-checkpoint runtime reduction. A manifest records source paths and file hashes. No additional training, diagnostic evaluation or timing experiment was performed to assemble this manuscript.